Bogazici University

Formal Verification of NTT-Based Fast Polynomial Multiplication in Lean

CompPoly Verified zkEVM

Salih Erdem Koçak, Doran Pamukçu

Advisor: Doğan Ulus   Department of Computer Engineering, Boğaziçi University

Motivation

Polynomial multiplication is a basic operation behind computable algebra and proof-system code. This project adds a faster path to CompPoly and proves that the executable result is the same polynomial as ordinary multiplication.

Method	Main operation	Growth
Naive multiplication	coefficient convolution	$O(n^2)$
NTT multiplication	transform, pointwise product, inverse	$O(n \log n)$

Implementation Surface

parameters — **Domain**: size, root of unity, inverse root

algorithm — **Forward / Inverse Transform**: iterative radix-2 butterfly implementation

pipeline — **FastMul.Raw**: forward transform, pointwise multiplication, inverse transform

public API — **FastMul**: canonical `CPolynomial` result

The public operation is small; most of the work is in connecting executable array code to mathematical transform specifications and then to polynomial multiplication.

NTT Multiplication

coefficient inputs — **evaluation domain** — **coefficient output**

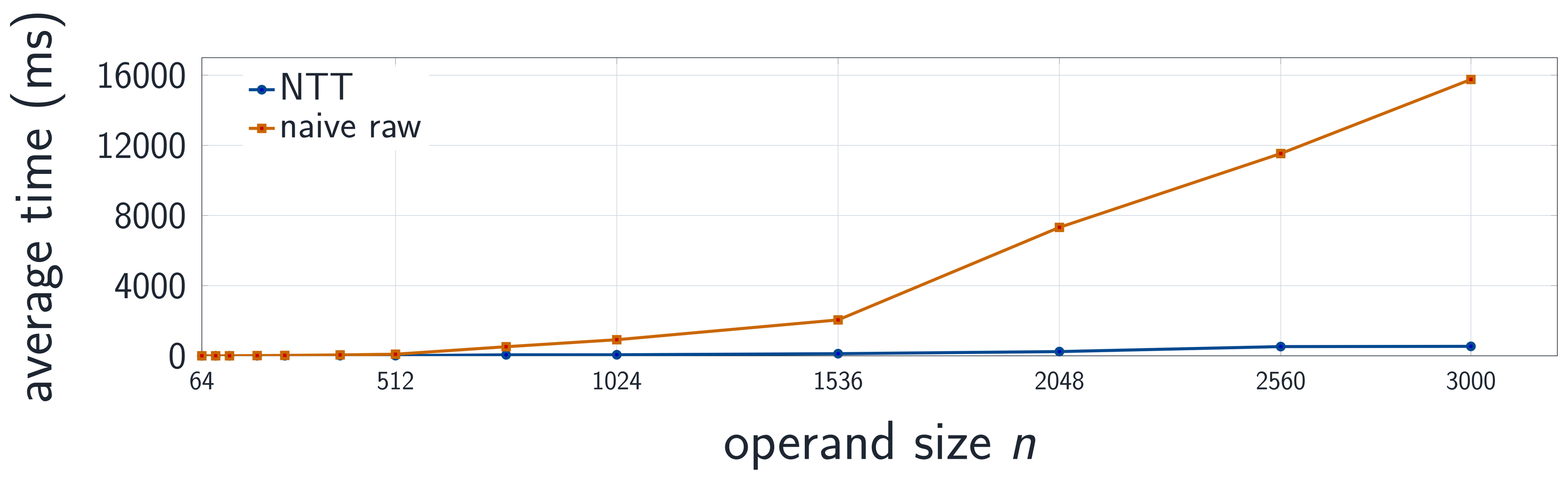
$p(x) = a_0 + a_1 x + \cdots$ → **NTT** → $\widehat{p} = [p(\omega^i)]_{i<n}$

$q(x) = b_0 + b_1 x + \cdots$ → **NTT** → $\widehat{q} = [q(\omega^i)]_{i<n}$

pointwise product: $\widehat{c} = \widehat{p} \odot \widehat{q}$, $\widehat{c}_i = \widehat{p}_i \widehat{q}_i$ → **inverse NTT** → $c(x) = p(x)q(x)$, $c_k = \sum_{i+j=k} a_i b_j$

The transform evaluates at powers of a root of unity. Multiplication becomes pointwise in the evaluation domain, then the inverse transform returns the coefficient representation.

Benchmark Behavior



Manual KoalaBear benchmark sweep from `tests/CompPolyTests/Univariate/NTT/Benchmark.lean`. Times are illustrative and machine-dependent.

Main Theorem

```
theorem fastMulImpl_eq_mul
    (D : Domain R)
    (p q : CPolynomial R)
    (hfit : Domain.fits D p.val q.val) :
    fastMulImpl D p q = p * q
```

This is the public correctness statement for the multiplication implementation. The precondition `Domain.fits D p.val q.val` says that the selected NTT domain is large enough for the ordinary convolution, so the transform pipeline has no cyclic wraparound.

Proof Architecture

code — **Executable loops**: bit reversal and butterfly stages over arrays

refinement — **Implementation refinement**: `forwardImpl_correct` and `inverseImpl_correct`

spec — **Direct NTT specifications**: root-of-unity summation formulas for forward and inverse transforms

algebra — **Evaluation-domain algebra**: pointwise products match evaluation of $p(x)q(x)$

shape — **No wraparound under `Domain.fits`**: the chosen domain is large enough for the convolution length

result — **Public theorem**: `fastMulImpl_eq_mul`: canonical fast multiplication equals $p * q$

References

1. Trieu, A., "Formally Verified Number-Theoretic Transform." *IACR Communications in Cryptology*, vol. 2, no. 4, Jan. 2026. DOI: 10.62056/ahbn-4tw9.
2. Capretta, V., "Certified Fast Fourier Transform." *Theorem Proving in Higher Order Logics*, TPHOLs 2001. Springer, 2001.
3. Ballarin, C., "Fast Fourier Transform." *Archive of Formal Proofs*, Oct. 2005. isa-afp.org/entries/FFT.html.
4. Truth Research ZK, "AMO-Lean", 2026. github.com/lambdaclass/amo-lean.

Code and Repository

Merged PR #174

CompPoly Repository